# SISTEMAS OPERATIVOS I

## Práctica 3: Uso de señales

## Objetivo

Utilización de llamadas al sistema para el envío y procesamiento de señales entre procesos.

## Información útil para la práctica

UNIX define un conjunto de señales que pueden enviarse desde un proceso hacia otro el cual, al recibirla, reaccionará de algún modo programado, es decir, las "capturará" . Existe un conjunto de señales ya predeterminadas en UNIX; el fichero *signal.h* las enumera, y se pueden ver en el volumen 7 del manual de *signal* (con *man 7 signal*). Por ejemplo, la señal 9 (denominada SIGKILL) se utiliza para finalizar un proceso o la señal 2 (SIGINT) es la asociada a la interrupción desde teclado CtrlC. Otro grupo de señales, SIGUSR1 y SIGUSR2, no tienen asignada ninguna reacción por defecto, ya que están pensadas para que las programe el usuario.

POSIX ofrece una función para enviar señales a un proceso denominada *kill* (ver el volumen 2 del manual de esta función). Para cada señal existe un modo de actuar por defecto, pero para algunas se puede cambiar usando las llamadas al sistema *signal* o *sigaction*. La llamada al sistema *sigaction* es más versátil y permite más funcionalidades que *signal*.

## Enunciado

1. Busca información sobre las funciones comentadas.

2. Haz un programa que se quede esperando a que introduzcas un dato (por ejemplo con la función *scanf*). Termina el programa enviando la señal SIGKILL desde consola con el comando *kill* (ver el volumen 1 del manual de este comando). Comprueba que puedes enviar esta señal utilizando el comando kill desde otra consola diferente de la que has usado para lanzar el proceso.

3. El siguiente código esboza un programa que utiliza las señales SIGUSR1 y SIGUSR2 entre un proceso padre y otro hijo con una única función personalizada de captura de estas señales. Analiza el código, complétalo según se indica en las líneas comentadas y comprueba su funcionamiento.

```
/* Incluye las librerias necesarias*/
  static void gestion(int); /*Declaracion de la funcion de gestion de señales recibidas*/
  main() {
/* Declara las variables necesarias*/
    if (signal(SIGUSR1, gestion) == SIG_ERR) printf("Error al crear gestor 1\n");
    if (signal(SIGUSR2, gestion) == SIG_ERR) printf("Error al crear gestor 2\n");
    padre=getpid();
    if ((hijo = fork()) == 0) {      /*Trabajo del hijo*/
        kill(padre,SIGUSR1);         /*Envia señal al padre*/
        for (;;);                    /*Espera señal del padre indefinidamente*/
    }
    else {                           /*Trabajo del padre*/
/* Completa el código del padre para enviar las señales SIGUSR2 y SIGTERM al hijo,
   y espera a que acabe con wait */
    }
  }
```

```
static void gestion(int numero_de_senhal) { /* Funcion de gestion de señales*/
  switch (numero_de_senhal) {
  case SIGUSR1:                              /*Entra señal SIGUSR1*/
      printf("Señal tipo 1 recibida. Soy %d\n",getpid()); break;
/*Completa para el resto de las señales usadas*/
  return; }
```

4. Programa un proceso con un lazo infinito y que sustituya la acción asociada CtrlC por la impresión de la frase: "Señal SIGINT atrapada y desactivada". Después de recibir 3 veces CtrlC, en las siguientes ocasiones debe preguntar al usuario si se debe restaurar el funcionamiento habitual y, en caso de que la respuesta sea afirmativa, restaurarlo. Para ello utiliza la opción SIG_DFL. Todas estas funcionalidades deben formar parte de la función que captura la señal.

5. Escribe un programa en C, que al ser ejecutado da lugar al proceso P, con llamadas a POSIX para crear dos procesos hijos, denominados H1 y H2. El proceso padre P debe esperar a la finalización del primer hijo H1 con la llamada *waitpid*. El segundo hijo H2 debe finalizar al primero (H1) con *kill* sin preguntarle al usuario. Asegúrate que el proceso padre (P) y el segundo hijo (H2) terminan normalmente.

6. Haz un programa que restaure la acción para la señal SIGINT al estado predeterminado una vez que el manejador de señal haya sido llamado. Usa la llamada al sistema *sigaction* para capturar la señal SIGINT y mostrar un mensaje indicando que la señal ha sido recibida. Comprueba que el proceso termina después de recibir la señal por segunda vez.

7. Haz un programa que cada vez que reciba la señal SIGUSR1 cree un proceso hijo. Añade manejadores usando la llamada al sistema *sigaction* para contar cada segundo el número de procesos creados y para procesar la señal SIGUSR1. Si el número de procesos es mayor que 5 el programa termina. El padre permanecera en *pause* hasta recibir una señal. Los hijos deben suspenderse a si mismos usando la función *raise*. El programa no puede almacenar los PIDs de los hijos. Antes de terminar, el padre debe asegurarse que todos los hijos hayan terminado. **ENTREGABLE 1**.

   *Nota 1: Los manejadores de señales deben limitarse estrictamente a registrar la recepción del evento actualizando variables globales de tipo volatile sig_atomic_t.*

   *Nota 2: Toda la lógica pesada del programa debe ejecutarse de forma segura dentro del bucle principal del proceso padre tras desbloquearse de pause(). Para consultar la lista exhaustiva de funciones reentrantes seguras (async-signal-safe), revisa la sección 7 del manual (*`man 7 signal-safety`*).*

8. Haz un programa P que capture las señales SIGUSR1 y SIGUSR2 con las siguientes características. **ENTREGABLE 2**.

   - El proceso P captura las señales SIGUSR1 y SIGUSR2 con la llamada al sistema *sigaction*, de tal modo que el manejador de las señales SIGUSR1 y SIGUSR2 simplemente debe mostrar en pantalla un mensaje indicando qué señal se ha recibido.
   - A continuación P crea un proceso hijo H1, y el proceso H1 crea un proceso nieto N1. El proceso P espera a que el hijo H1 termine y muestra en pantalla el valor del código devuelto a través del comando *exit* de ese hijo. Utiliza diferentes códigos en el exit de H1 y N1 para que puedan ser identificados.
   - El proceso hijo H1 envía la señal SIGUSR1 al proceso P y espera a que termine N1. El proceso nieto N1 envía la señal SIGUSR2 al abuelo, espera 5 segundos y termina.
   - Además de lo indicado anteriormente, el proceso padre P debe bloquear la señal SIGUSR1 durante un tiempo antes de capturarla.

- Para poder ver el efecto del bloqueo, P debe bloquear la señal SIGUSR1 antes de que el hijo H1 se la envíe y desbloquearla después de que el nieto haya enviado la señal SIGUSR2. Para ello, una vez que se ha bloqueado la señal SIGUSR1, se debe dormir el proceso padre durante un tiempo suciente para que el nieto envíe la señal SIGUSR2 antes de que se despierte y desbloquear la señal SIGUSR1 después de despertarse. Antes de desbloquear la señal, el padre debe comprobar que SIGUSR1 está pendiente (bloqueada). Como resultado, el padre debería procesar la señal SIGUSR2 antes que la SIGUSR1 a pesar de haberla recibido más tarde. Utiliza la llamada al sistema *sigprocmask* para bloquear la señal y las llamadas *sigpending* y *sigismember* para comprobar si están bloqueadas.
- Todos los procesos involucrados deben **imprimir en consola comentarios exhaustivos** sobre cada una de las acciones que realizan, identificándose y mostrando la hora para comprobar de una manera clara que la secuencia de acciones es la esperada.

*Nota 1: En la estructura tipo sigaction es conveniente activar el flag SA_RESTART, que fuerza el reiniciado de ciertas llamadas del sistema cuando sean interrumpidas por un gestor de señal; en caso contrario la llamada al sistema se interrumpiría cuando el proceso recibe la señal.*

*Nota 2: Se recomienda que primero hagas el programa sin bloquear la señal SIGUSR1, y cuando funcione correctamente introduzcas el bloqueo y la comprobación de que está pendiente.*